

Lunalink (SIH26166) — The Complete Plain-English Companion

Everything in `PRESENTATION_GUIDE.md`, decoded: every acronym, every formula, every design decision, explained from first principles.

0. How to read this document

The original guide is written **for someone who already knows the field** — it's built so a team member can defend any claim to a judge in seconds, using exact file and function names. That's great for speed, but it assumes you already know what RANSAC, ONNX, RLS, or a Lambertian reflectance model *are*.

This companion does the opposite: it assumes **nothing**. Every section below mirrors the original guide's structure (Part 1 → Part 4), but every piece of jargon is unpacked the first time it appears, formulas are broken into "what each symbol means" + "why this is the right equation," and each design decision is explained in terms of the *problem* it solves, not just the *answer* the team picked.

Section 1 gives you the 60-second plain-English pitch. **Section 2** is a master glossary — if a term ever stumps you while reading anything else in this document, it's defined there. **Sections 3–6** walk through the original guide's four parts in order, line by line, in plain English.

1. The 60-second plain-English pitch

What the project actually does: it takes a photograph of the Moon's surface taken by an Indian spacecraft (Chandrayaan-2's OHRC camera), and:

Finds the same patch of ground in an independent American satellite's photograph (NASA's LRO spacecraft) — proving the two space agencies' images agree with each other, automatically, with no human pointing at matching landmarks.

Reconstructs a 3-D terrain model — hills, crater bowls, ridges — from *just the shading in a single 2-D photo*, the way your brain infers that a photo of a bowl is concave just from how the light falls on it.

Finds craters automatically and estimates how deep each one is, using nothing but the length of the shadow it casts.

Displays all of this in a live 3-D model you can rotate, with an AI voice-over that narrates the numbers — and it accepts *your own* uploaded photo and does the same pipeline on it, live, in front of judges.

Why this is hard: the two satellite photos are taken by different cameras, at different times, with the sun in a different position in the sky. A shadow that looks like a crater rim in one photo can look like a flat patch in the other. Most simple image-matching algorithms get fooled by this lighting change. Lunalink's core research

contribution is a matching system that is deliberately trained to *ignore* lighting differences and focus on the actual shape of the terrain underneath.

Why the tech choices matter: the team was explicit that every technology choice was made to answer one question — “does this solve *our* problem, or is it just industry-standard for problems we don’t have?” — and the guide spends Part 1 defending exactly that reasoning, one tool at a time.

2. Master glossary

Organized by category. Skim it once, then use it as a reference while reading Sections 3–6.

2.1 Web frontend terms

Term	Plain-English meaning
Frontend	The part of the app that runs in the user’s browser — what you see and click.
Backend	The part of the app that runs on a server, does the heavy computation, and answers requests from the frontend.
React / Vue / Angular	The three most popular <i>JavaScript frameworks</i> — pre-built toolkits for building web pages whose content changes dynamically (e.g., a live chat window). They all solve the same core problem: keeping what’s on-screen in sync with changing data.
Component framework	A tool (like React) that lets you build a web page out of small, reusable, self-updating pieces (“components”) instead of writing one giant page by hand.
Vanilla JS	Plain JavaScript with <i>no</i> framework on top — “vanilla” as in “not flavored with anything extra.”
ES modules	A standard, built-into-every-browser way of splitting JavaScript code across multiple files and having them <code>import</code> / <code>export</code> code from each other — without needing a separate tool to combine them first.
Import map	A small block of configuration in an HTML page that tells the browser “when code says <code>import Three from 'three'</code> , actually fetch it from <i>this</i> URL” — lets you use library names like a framework would, with zero build tooling.
CDN (Content Delivery Network)	A network of servers spread around the world that host a copy of a public file (like the Three.js library) so it loads fast from wherever the user is, without you having to host it yourself.

Term	Plain-English meaning
Build step / bundler (Webpack, Vite)	A tool that takes your many source code files and <i>compiles</i> them into an optimized bundle before the browser ever sees them. Vite is a newer, faster alternative to the older Webpack. Both add complexity, config files, and a <code>node_modules</code> folder full of dependencies.
HMR (Hot Module Replacement)	A developer convenience where a bundler updates the running page instantly when you save a code change, without a full page reload. Only relevant if you have a build step to begin with.
<code>node_modules</code>	The folder where all of a JavaScript project's downloaded dependencies live — often huge (hundreds of MB) and a common source of "supply-chain" security risk, since you're trusting every package (and every package <i>those</i> packages depend on).
<code>.gitignore</code>	A file that tells Git (the version-control system) which files/folders to <i>never</i> track or upload — e.g., secrets, or <code>node_modules</code> .
Supply-chain risk	The security risk that comes from depending on code you didn't write and haven't audited — if any dependency (or a dependency's dependency) is compromised, your app inherits that vulnerability.

2.2 3-D graphics terms

Term	Plain-English meaning
Three.js	A JavaScript library that makes it possible to draw and animate 3-D scenes inside a web browser, using the browser's built-in GPU-accelerated graphics (WebGL) without you having to write raw graphics code.
Mesh	The 3-D "skeleton" of a shape — a collection of points (vertices) connected into flat triangles that approximate a curved surface.
<code>PlaneGeometry(1000, 1000, 191, 191)</code>	Three.js code that creates a flat grid 1000×1000 units in size, subdivided into 191×191 segments — i.e., a fine grid of squares (each made of 2 triangles) that can then be pushed up and down to form terrain.
Vertex (plural: vertices)	A single 3-D point (an x, y, z coordinate) — the corner of a triangle in a mesh.
Triangle	The basic building block of almost all 3-D graphics — every curved surface you see in a 3-D scene is actually thousands of tiny flat triangles.

Term	Plain-English meaning
Y-displaced	Moved up or down along the vertical (Y) axis — here, each grid point's height is pushed up/down according to the computed terrain elevation at that spot, turning a flat grid into a bumpy landscape.
DEM (Digital Elevation Model)	A grid of numbers where each number is the height of the ground at that location — essentially a height-map, the raw data a 3-D terrain mesh is built from.
Wireframe	A rendering mode that draws only the edges of every triangle (not the filled surface), so you can see the underlying mesh structure.
Contour lines / marching squares	Lines connecting all points at the same elevation (like on a topographic map). "Marching squares" is the specific algorithm used to compute exactly where those lines should be drawn (explained fully in Section 6).
Vertical exaggeration	A multiplier applied to height data to make relief more visually obvious — real lunar terrain relief is often subtle relative to its horizontal extent, so a raw 1:1 render can look almost flat. The guide stresses this multiplier is <i>computed</i> , not guessed, so the visualization doesn't overstate the actual relief.
OrbitControls	A standard Three.js add-on that lets a user click-drag to rotate, zoom, and pan around a 3-D scene with the mouse (or touch, on mobile).

2.3 Backend / API terms

Term	Plain-English meaning
FastAPI	A modern Python framework for building web APIs (the backend "brain" that the frontend talks to) — popular for being fast to write in and fast to run.
API (Application Programming Interface)	A defined set of URLs/requests a program can send to get data or trigger an action — e.g., <code>GET /match/{scene_id}</code> is an API endpoint that returns match results for a given scene.
Async / <code>async def</code> vs sync <code>def</code>	Two ways to write a function that handles a request. A synchronous function runs start-to-finish, blocking everything else until it's done. An asynchronous function can <i>pause</i> itself while waiting on something slow (like a network request) so the program can do other work in the meantime, then resume.
Event loop	The single "traffic controller" inside an async program that decides what runs next; if any one task blocks it for too long without yielding, <i>everything</i> on that event loop stalls.

Term	Plain-English meaning
Threadpool	A small pool of worker threads a program can hand slow/blocking tasks to, so the main event loop doesn't get stuck waiting for them. FastAPI's underlying engine, Starlette , automatically runs ordinary (<code>sync def</code>) functions in a threadpool of about 40 workers.
CPU-bound vs GPU-bound vs I/O-bound	Describes <i>what kind of resource</i> a task is limited by. CPU-bound = limited by processor speed (e.g., heavy math). GPU-bound = needs a graphics/AI accelerator chip to be fast. I/O-bound = limited by waiting on external things like network calls or file access, not computation.
Concurrency vs parallelism	Concurrency = <i>managing</i> many tasks that are all in progress (possibly by interleaving them). Parallelism = actually <i>running</i> multiple tasks at the exact same instant on multiple processor cores. A threadpool gives you a practical form of both for CPU-bound work.
Caching / cache-aside	Storing the result of an expensive computation so that if the same request comes again, you return the saved answer instead of recomputing it. "Cache-aside" is a specific pattern: check the cache first; if it's missing ("a miss"), compute the real answer and <i>then</i> save it to the cache for next time.
In-memory dict	The simplest possible cache — a Python dictionary living in the server's RAM. Fast, but wiped out if the server restarts, and not shared between multiple server instances.
TTL (Time To Live)	An expiration timer on a cached value — after this much time, the cache entry is automatically thrown away and the next request recomputes it fresh.

2.4 Machine learning / AI terms

Term	Plain-English meaning
PyTorch	A widely used Python library for building and training neural networks (the "engine" used to <i>train</i> the model).
ONNX (Open Neural Network Exchange)	A standard file format for a <i>trained</i> neural network that lets you run it in a program other than the one that trained it (here: trained in PyTorch, run inside the Python backend via a lightweight "ONNX Runtime" engine, without needing all of PyTorch installed on the server).
<code>torch.onnx.export</code>	The specific PyTorch function that converts a trained PyTorch model into an ONNX file.

Term	Plain-English meaning
Opset (operator set) version	ONNX's own internal versioning system for which mathematical operations ("ops") are supported — like specifying which edition of a rulebook the exported file follows, so any ONNX-compatible program knows how to read it correctly.
Dynamic batch axis	A flag telling the exported model "the number of items processed at once doesn't have to be fixed — it can vary," so the server can feed it 1 image patch or 256 at once without re-exporting the model.
Quantization	A technique to shrink and speed up a model by using lower-precision numbers (e.g., 8-bit integers instead of 32-bit decimals) — saves memory/speed but can lose some accuracy. Lunalink's export explicitly does not do this.
Pruning	A technique that removes less-important connections from a trained neural network to make it smaller/faster — also not done here, for the same accuracy-preservation reason.
float32	A standard 32-bit way of representing decimal numbers in a computer — "the normal precision." The guide stresses the model is float32 going in and float32 coming out, so no precision was thrown away in conversion.
<code>onnxruntime</code>	The lightweight software that actually <i>runs</i> (executes) an ONNX model — separate from PyTorch, and much smaller to install on a server.
<code>ExecutionProvider</code>	The specific hardware backend <code>onnxruntime</code> uses to run the model — <code>CPUExecutionProvider</code> means "run on the regular processor," as opposed to a <code>CUDAExecutionProvider</code> , which would use an NVIDIA GPU.
CUDA	NVIDIA's platform that lets software run computations on a graphics card (GPU) instead of the regular processor (CPU) — GPUs are much faster for the kind of massively-parallel math neural networks need, but require specific hardware and drivers to use.
CNN (Convolutional Neural Network)	A type of neural network especially good at processing images, built from layers that each scan small patches of the image looking for patterns (edges, textures, shapes), stacking up into more complex pattern recognition layer by layer.
Conv layer / channels	One layer of a CNN. "16→32→64→128 channels" means the network starts by learning 16 different simple patterns, then combines those into 32 more complex patterns, then 64, then 128 — each layer building on the last.
ReLU	A very simple, standard "activation function" used between neural network layers — it just zeroes out any negative number and leaves positive numbers unchanged. It's

Term	Plain-English meaning
	what lets a neural network learn non-simple (non-straight-line) patterns.
MaxPool	A layer that shrinks an image by keeping only the strongest signal in each small local region — reduces size while keeping the most important information.
AdaptiveAvgPool	Similar idea, but averages instead of taking the max, and automatically adjusts to produce a fixed output size regardless of input size.
Linear layer	A layer that does a straightforward weighted combination of all its inputs — often the final layer that turns a big pile of extracted features into a compact output vector.
Embedding	A list of numbers (a vector) that represents something complex (like an image patch) in a compact, comparable form — the whole point of training is to make embeddings of <i>similar</i> things end up numerically close to each other.
L2-normalize / L2 norm	"L2 norm" is a fancy name for a vector's length (computed as the square root of the sum of its squared components — ordinary Euclidean distance from the classic Pythagorean theorem, just extended to more than 2 or 3 dimensions). "L2-normalizing" rescales a vector so its length is exactly 1, which makes comparing two vectors' <i>directions</i> (rather than sizes) mathematically clean.
Triplet loss	A training technique that shows the network three examples at once: an anchor , a positive (something that should be considered similar to the anchor), and a negative (something that should be considered different). The network is penalized whenever the negative ends up closer to the anchor than the positive does.
Margin	In triplet loss, how much <i>closer</i> the positive must be than the negative before the network is considered "confident enough" — a safety buffer, not just barely correct.
Adam optimizer	A widely used algorithm that automatically adjusts a neural network's internal numbers ("weights") during training, based on the errors it's making, so it gradually gets better.
Learning rate (lr)	How big a step the optimizer takes each time it updates the network's weights — too high and training becomes unstable; too low and training is painfully slow.
Batch	A small group of training examples processed together at once (rather than one at a time), which is both faster and produces a smoother, more reliable training signal.
Epoch	One complete pass through the entire training dataset.

Term	Plain-English meaning
NumPy / PIL	Two standard Python libraries — NumPy handles fast numerical array math, PIL ("Pillow") handles loading/saving/manipulating image files.
Kaggle / T4	Kaggle is a free online platform for running data-science code; a "T4" is a specific model of NVIDIA GPU that Kaggle offers for free, useful for training a bigger version of the model faster than a laptop CPU could.
Self-supervised learning	Training a model <i>without</i> needing humans to manually label data — here, the "labels" (which patches are similar/different) are generated automatically by the data-creation process itself, not hand-annotated.
Synthetic data	Artificially generated training data (here: computer-generated fake crater terrain), used instead of, or alongside, real photographs — useful when real labeled examples are scarce.
fBm (fractional Brownian motion) / octaves	A mathematical technique for generating realistic-looking random rough terrain by layering multiple scales of randomness on top of each other ("octaves" = layers, from coarse bumps down to fine texture), the same core idea used in a lot of computer-graphics terrain generation.
Albedo	How reflective/bright a surface naturally is, independent of lighting — two patches can be lit identically but look different in brightness purely because one material reflects more light than the other.

2.5 Computer vision & matching terms

Term	Plain-English meaning
SIFT (Scale-Invariant Feature Transform)	A classic, well-established computer-vision algorithm that finds distinctive "keypoints" in an image (corners, blobs, textured spots) and describes each one with a numeric fingerprint, designed so the <i>same</i> real-world point can be re-recognized even if the image is scaled, rotated, or has some lighting change.
Keypoint	One of these distinctive, easily-relocatable points SIFT finds in an image — think of them as natural "landmarks" a computer can latch onto.
DoG (Difference of Gaussians)	The specific mathematical trick SIFT uses internally to find keypoints — it blurs an image by different amounts and looks at the <i>difference</i> between blur levels, which highlights blob-like features at various sizes.
Sub-pixel refinement (Taylor	After roughly locating a keypoint on the image's pixel grid, this technique nudges its estimated location to a fraction of a pixel more precisely, using a local mathematical

Term	Plain-English meaning
expansion)	approximation (a Taylor expansion) — extra precision that matters a lot when tiny position errors would otherwise accumulate.
Lowe ratio test	A filtering rule (named after SIFT's inventor, David Lowe) for deciding if a proposed match between two keypoints is trustworthy: compare the best match's distance/dissimilarity score to the <i>second</i> -best candidate's score. If the top match isn't convincingly better than the runner-up, it's discarded as too ambiguous to trust.
Cosine similarity	A way of measuring how similar two vectors are by looking at the <i>angle</i> between them rather than their raw values — 1.0 means pointing in exactly the same direction (very similar), 0 means unrelated, useful here for comparing the learned 128-D embeddings.
Mutual nearest neighbor	A stricter matching rule: keypoint A and keypoint B are only accepted as a match if A's closest match is B and B's closest match is also A (i.e., they each independently pick each other as their best partner), which filters out a lot of false matches.
RANSAC (Random Sample Consensus)	An algorithm for finding a reliable pattern (here: a geometric transformation) inside noisy data that contains some outliers (wrong matches). It repeatedly picks small random subsets of the data, fits a candidate model to each subset, and keeps whichever candidate the most other data points agree with.
Inlier / outlier	An inlier is a data point (here: a matched keypoint pair) that's consistent with the model RANSAC settled on. An outlier is a point that doesn't fit — likely a wrong/spurious match.
Homography	A mathematical transformation (represented as a 3×3 matrix) that maps points from one flat image onto corresponding points in another, accounting for rotation, scaling, translation, and perspective — the standard way to describe "how image A's viewpoint relates to image B's viewpoint" when both are looking at the same flat-ish scene.
DLT (Direct Linear Transform)	A specific, standard algorithm for computing that homography matrix directly from a set of matched point pairs, using linear algebra.
4-point / minimal sample	A homography has enough "unknowns" that it can be uniquely computed from just 4 matched point pairs — the smallest possible sample RANSAC needs per trial round.
Reprojection error / RMSE (Root Mean Squared Error)	After fitting a homography, you can check how well it actually explains the data: take each matched point, apply the homography, and see how far off the predicted location is from the real one (this gap is the "reprojection error"). RMSE combines all these individual errors into one overall accuracy number by squaring each, averaging, and taking the square root (which keeps the number in the same units as the original error, e.g., pixels, and penalizes large errors more heavily than small ones).

Term	Plain-English meaning
Confidence (RANSAC parameter, p)	How sure you want RANSAC to be that it found a genuinely correct model, not one that got lucky on a random outlier-heavy sample — a higher confidence target (e.g., 0.99 = 99%) means RANSAC will run more trial iterations to be safe.
Iteration budget / derived k	How many random-sample trials RANSAC needs to run to hit that confidence target — explained fully with its formula in Section 6.
CLAHE (Contrast Limited Adaptive Histogram Equalization)	An image-processing technique that boosts local contrast (making faint details more visible) across different regions of an image independently, while capping how much any one region can be boosted, so noise doesn't get amplified out of control. Used here to make faint terrain texture easier for SIFT and the learned model to pick up on.
Fourier-Mellin transform	A mathematical technique (explained fully in Section 6) for estimating the rotation, scaling, and translation between two images purely from their frequency content — used here as a secondary sanity check/diagnostic, not the primary matching method.
Phase correlation	A frequency-domain technique for finding how much one image is shifted relative to another, by comparing the <i>phase</i> (positional) information in their Fourier transforms rather than comparing raw pixel brightness.
NCC (Normalized Cross-Correlation) / template matching	A classic technique for finding where a small reference image ("template") best matches inside a larger image, by sliding the template across every position and scoring how well it lines up, normalized so the score isn't thrown off by differences in overall brightness/contrast between the two images.
Pearson correlation	A standard statistical measure (from -1 to +1) of how linearly related two sets of numbers are — used here to quantify how well two independently-derived terrain estimates agree with each other.

2.6 Math & signal-processing terms (for Part 4)

Term	Plain-English meaning
Lambertian reflectance	A simple, standard model of how a matte (non-shiny) surface reflects light: brightness depends only on the angle between the surface and the light source, not on where the <i>viewer</i> is standing. It's the physical assumption that makes "brightness → slope" math possible.
Partial derivative ($\partial h / \partial x$)	How fast a quantity (here, height $\boxed{h}$) changes as you move in one specific direction (here, the x-direction), holding everything else fixed — in plain terms, "the local steepness of the terrain, measured along one axis."

Term	Plain-English meaning
Gradient / slope field (p, q)	The pair of partial derivatives (steepness in the x-direction and steepness in the y-direction) at every point — together they describe which way, and how steeply, the ground tilts at each pixel.
Minimum-norm solution	When a math problem doesn't have enough information to pin down one unique answer (there are many possible answers that all fit the data equally well), a "minimum-norm" solution is the specific one among all valid options that is <i>smallest</i> in magnitude — a principled, conservative way to pick an answer rather than guessing arbitrarily.
FFT (Fast Fourier Transform)	An extremely efficient algorithm for converting a signal (or image) between "how it looks in space/time" and "what frequencies (patterns of repetition) it's made of." Used throughout this project both for actual computation (like solving the Poisson equation below) and for analysis (like the Fourier-Mellin registration).
Poisson equation / Poisson solve	A classical equation from physics and mathematics relating a quantity's <i>curvature</i> to some driving source — here, relating a terrain surface's second derivatives (curvature) back to the <i>slope field</i> computed from shading, so that integrating the slopes reconstructs the actual height surface. Solving it via FFT is a standard, fast numerical technique.
DC guard / DC component	In signal processing, the "DC" (Direct Current, borrowed from electrical engineering) component of a Fourier transform is its zero-frequency term, essentially the overall average value of the signal. It's mathematically undefined/unstable in this particular Poisson-solving formula (you'd be dividing by zero), so it's manually clamped to a safe placeholder value — the "guard."
Hanning window	A standard smooth tapering function applied to signal edges before doing Fourier analysis, which reduces artificial jumps/artifacts that would otherwise appear at the boundaries of a finite image.
Radiometric calibration	Adjusting a reconstructed brightness/height value onto a physically sensible absolute scale, since raw shading data alone can only tell you <i>relative</i> shape, not the true absolute height.
RMS (Root Mean Square)	Take a set of values, square each one, average the squares, then take the square root — a standard way to summarize the "typical size" of a set of numbers regardless of their sign (this project uses it both for average slope steepness and for reprojection error).

Term	Plain-English meaning
Hough transform (circle detection)	A classic computer-vision algorithm for detecting simple geometric shapes (here, circles — i.e., craters) in an image, even when the shapes are noisy or partially obscured, by having every edge pixel "vote" for which circles it could plausibly be part of, and keeping the circles with the most votes.
Bilinear interpolation	A simple, standard way to estimate a value at a position that falls <i>between</i> known grid points, by blending the four nearest known values, weighted by how close the target position is to each.
Log-polar coordinates	Re-expressing an image using distance-from-center (on a logarithmic scale) and angle-from-center, instead of ordinary x/y — this specific re-mapping is what turns "rotation and scaling" in the original image into simple "shifting" in the new coordinate system, which is much easier to detect.
Mutual information	A statistical measure of how much knowing one variable tells you about another — unlike simple correlation, it can capture <i>any</i> kind of statistical relationship, not just straight-line ones, which makes it useful for comparing images from very different sensor types where brightness relationships aren't simply proportional.
Jensen's inequality	A general mathematical fact about "concave" functions (functions that curve like a frown, $\cap$) — it's the formal reason the mutual information formula is guaranteed to always be zero or positive, never negative, which is what makes it usable as a valid "how related are these" score.
Low-pass filter	A filter that keeps the slow, large-scale patterns in data (or an image) and removes the fast, small-scale, noisy variation — literally, it lets low frequencies "pass" through and blocks high frequencies.
Coefficient / frequency-domain representation	When you run an FFT on an image, you get back a grid of numbers ("coefficients") describing how strongly each specific repeating pattern (frequency) is present in the image — keeping only a handful of the <i>low-frequency</i> coefficients and discarding the rest is exactly what a low-pass filter does.
Scan-line artifact	A repeating stripe-like pattern of noise sometimes introduced by how a satellite camera physically scans/captures an image line by line — because it's so regular/repetitive, it shows up concentrated at one specific spot in the frequency-domain representation, making it easy to surgically remove there.
Cramér-Rao bound	A fundamental result from statistics that sets a hard floor on how precisely <i>any</i> method could possibly estimate something from noisy/limited data — it's invoked here to argue that once matching accuracy stops improving, that's not a bug in the code, it's

Term**Plain-English meaning**

because the *image itself* physically doesn't contain any more usable information to extract.

2.7 Infrastructure / cloud / deployment terms

Term**Plain-English meaning****Vercel**

A cloud hosting service specialized in deploying *static* websites (and frontend frameworks) fast, globally, for free at small scale.

Static site

A website made of files that don't change based on who's asking or need server-side computation to generate — just plain HTML/CSS/JS files served as-is.

`vercel.json` /
`outputDirectory`

A configuration file that tells Vercel exactly which folder in the project contains the files to actually publish.

Render

A cloud hosting service that runs full backend applications (unlike Vercel, it can run a persistent Python process, not just static files).

Container / web service

A packaged, self-contained way of running an application (with all its dependencies bundled) so it runs identically regardless of the underlying server — a "web service" here just means a long-running program that listens for and responds to web requests.

`render.yaml`
(Blueprint)

A configuration file that describes, in one place, exactly how to build and run the backend on Render — what command installs dependencies, what command starts the server, etc. — so a fresh deployment is fully automatic and reproducible.

`uvicorn`

The actual program that runs a FastAPI application and listens for incoming web requests — FastAPI defines *what* to do with a request, uvicorn is the engine that actually receives it over the network.

`requirements.txt`

A plain-text list of every Python package the project depends on (and often, which exact version), so anyone (or any server) can install the identical set of dependencies with one command.

Health check

A simple endpoint (here, `/health`) that a hosting platform repeatedly pings to confirm the server is alive and responding, used to detect and restart a crashed/stuck server automatically.

`window.API_BASE` /
config.js

A single configuration value in the frontend code that stores the backend's web address — changing this one line is all that's needed to point the frontend at a

Term	Plain-English meaning
	different backend (local machine, Render, etc.).
CORS (Cross-Origin Resource Sharing)	A browser security rule that, by default, blocks a webpage from one website from making requests to a server on a <i>different</i> website/domain, unless that server explicitly says "requests from other origins are allowed." <code>allow_origins=["*"]</code> means "allow requests from any origin," a permissive setting appropriate for a public demo with no private user data.
Stateless	Describes a backend that doesn't remember anything about "who" is asking between requests — each request stands alone and carries everything needed to answer it, rather than relying on a remembered login session. Simpler to reason about and scale.
Cookies / sessions	The standard mechanism websites normally use to remember a logged-in user across multiple requests — deliberately <i>not</i> used here, since the system has no user accounts to track.
Cold start	The delay before a "sleeping" free-tier server wakes back up and is ready to respond, after sitting idle for a while — a known trade-off of using free hosting tiers.
HTTPS tunnel	A tool that takes a program running on a private machine (like someone's laptop) and gives it a temporary public internet address, so it's reachable from anywhere, without actually deploying it to a real hosting service — useful for live demos where you want your own machine's full processing power/data instantly available.

2.8 Database / storage / other integrations

Term	Plain-English meaning
Supabase	A hosted platform that provides a full managed PostgreSQL database plus extra developer conveniences (an automatic REST API, user authentication, file storage) without you having to run any database server yourself.
PostgreSQL ("Postgres")	A mature, powerful, open-source relational (table-based) database system — the actual database engine underneath Supabase.
RLS (Row-Level Security)	A database feature that lets you write rules controlling <i>which rows</i> of a table a given request is allowed to read or write — e.g., "anyone can read the <code>scenes</code> table, but only requests using a special key can write to the <code>jobs</code> table." This is enforced by the database itself, not just by the application code, which is a stronger security guarantee.

Term	Plain-English meaning
<code>jsonb</code> column	A PostgreSQL column type that stores flexible, structured JSON data (rather than forcing every field into its own rigid column) — used here for things like geographic footprints and transformation matrices, which don't fit neatly into simple rows/columns.
Service-role key	A special, highly-privileged access key for a Supabase project that bypasses RLS entirely — meant to be used only by trusted backend code, never exposed to the browser/frontend, since anyone holding it could read or write anything.
<code>.env</code> file	A file used to store secret configuration values (like API keys) <i>outside</i> of the actual source code, so secrets aren't accidentally committed to a public code repository.
GeoTIFF	A common file format for storing satellite/aerial imagery along with its precise geographic location data embedded in the file — files of this type are often very large (many megabytes to gigabytes).
PDS (Planetary Data System)	NASA's standard archival system/format for storing and publishing planetary science mission data (images, instrument readings, etc.) — the guide references PDS3 and PDS4, two different versions/generations of this standard's file format.
Upstash	A hosted, serverless-friendly version of Redis (see below) that you talk to over ordinary HTTP web requests instead of a persistent low-level connection — convenient for hosting environments (like serverless functions) that can't keep long-lived network connections open.
Redis	An extremely fast in-memory key-value data store, commonly used for caching, rate limiting, and temporary/fast-changing data — "in-memory" means data lives in RAM (very fast) rather than on disk, generally paired with some persistence/backup mechanism.
Key-value store	The simplest kind of database: you store and retrieve data by a unique "key" (like a filename), with no concept of tables, rows, or relationships.
<code>SET</code> / <code>GET</code>	The most basic Redis commands — store a value under a key, and retrieve it.
Sorted set	A Redis data structure that stores unique items each tagged with a numeric "score," automatically kept in sorted order by that score — useful here for tracking <i>when</i> each recent request happened, so old ones can be efficiently identified and dropped.
<code>ZADD</code> / <code>ZREMRANGEBYSCORE</code> / <code>ZCARD</code> / <code>PEXPIRE</code>	The specific Redis commands used to implement the rate limiter: add a timestamped entry, remove entries older than the current time window, count how

Term	Plain-English meaning
	many entries remain in the window, and set the whole key to auto-expire.
Sliding window vs fixed window (rate limiting)	Two ways to enforce "at most N requests per minute." A fixed window resets its counter at fixed clock boundaries (e.g., exactly on the minute), which can let a burst of requests slip through right at the boundary. A sliding window instead always looks at "the last 60 seconds from right now," giving a smoother, harder-to-game limit.
Rate limiting	Deliberately capping how many requests a system will accept in a given time period, to prevent abuse or to stay within the free-tier usage limits of a paid third-party service (like the Gemini AI API).
Lists / streams / pub-sub (Redis features, not used here)	Other Redis data structures/features for different jobs — ordered queues, event streams, and real-time broadcast messaging respectively. The guide notes deliberately <i>not</i> using these, since the simpler strings + sorted-set structures already cover every actual need — a "use only what you need" design choice.
Sentry	A hosted error-tracking service — when something goes wrong in the running app (frontend or backend), Sentry automatically captures the details and reports them, so the team can see and fix problems even from users/judges they weren't watching directly.
DSN (Data Source Name)	The specific address/identifier Sentry gives each project, used to tell Sentry's tracking code where to send its error reports. Frontend DSNs are meant to be public/visible in browser code — they only allow <i>sending</i> error reports, not reading any private data.
Breadcrumb (Sentry concept)	A small trail of "what just happened" events Sentry records leading up to an error, so when something does break, you can see the sequence of steps that led there, not just the final crash.
Gemini / GPT / Claude	Three different companies' large language model AI systems (Google, OpenAI, and Anthropic respectively) that can understand and generate natural-language text — used interchangeably in casual conversation as "which AI," though each has its own strengths/pricing.
"flash-lite" class model	Refers to a smaller, faster, cheaper tier within Google's Gemini model lineup — appropriate here because the task (summarizing some numbers into a short paragraph) doesn't need a large, expensive, slow model to do well.
Context window	How much text an AI model can "see"/consider at once when generating a response — irrelevant here because the input (a handful of computed metrics) is

Term	Plain-English meaning
	tiny compared to any model's limit.
Prompt-locked	A deliberate constraint on how the AI is used: it's instructed (and structurally restricted) to only rephrase/narrate numbers the pipeline already computed, rather than being free to invent or estimate its own numbers — a safeguard against the AI "hallucinating" plausible-sounding but false statistics.
429 (HTTP status code)	The standard web-server response code meaning "you've sent too many requests, slow down" — used by APIs (including Gemini's) to enforce their own rate limits on you.
Cooldown	A deliberate pause period the app enforces on itself after hitting a 429, so it doesn't keep hammering an already-rate-limited service and generating a flood of repeat error messages.
No-op ("no operation")	Code that, when a required configuration (like an API key) is missing, does <i>nothing</i> harmful and simply skips that feature gracefully, rather than crashing — the guide highlights that every optional integration (Supabase, Upstash, Sentry, Gemini) is built this way, so the whole app still works with zero of them configured.

3. Part 1 elaborated — the judge-facing Q&A

(Each numbered question below matches the original guide's numbering.)

Q1 — Why not React/Vue/Angular?

Plain-English answer: the team looked at what a component framework is actually *for* — managing lots of interdependent, constantly-changing pieces of on-screen state — and realized their app doesn't have that problem. The whole app has exactly one piece of shared state that matters (which lunar scene is currently selected), and *that* triggers a re-match. Pulling in React just to manage one variable would mean adding a whole build pipeline, a `node_modules` folder, and dozens of third-party dependencies (each one a small extra security risk) — for zero actual benefit. Instead, the team wrote plain (**vanilla**) JavaScript using the browser's own native module system (**ES modules**), loading only the one library they actually needed for something complex (Three.js, for 3-D graphics) straight from a CDN via an **import map**. The practical result: the entire frontend is a single `index.html` file with no build step at all.

Q2 — What does Vite give you over Webpack?

Plain-English answer: the honest answer is "neither, we use *no* bundler at all." Both Vite and Webpack solve the same underlying problem — taking many source files and compiling them into an optimized bundle before the browser runs them — and the team's point is that skipping this step entirely (using native **ES modules** +

import maps, both of which every modern browser already understands) is a strictly bigger advantage than picking a faster bundler. No build step means: nothing to configure, nothing that can break on demo morning, and the file you deploy is literally the exact file you wrote — there's no "did the build actually pick up my latest change?" uncertainty.

Q3 — Is the 3-D view real computed terrain, or just a nice-looking visual?

Plain-English answer: it's real, computed geometry, not a decorative animation. Concretely: the code builds a Three.js `PlaneGeometry` — a flat grid of **36,864 vertices** forming **72,962 triangles** — and then pushes every single vertex up or down based on an actual computed **DEM** (elevation grid, 192×192 values) produced by the shape-from-shading math described in Part 4. On top of the solid mesh, the code also draws a **wireframe** overlay (so you can see the underlying grid structure) and **8 contour lines** (computed via the marching-squares algorithm, Part 4) as actual 3-D line geometry sitting at the correct heights — not a flat 2-D topographic map pasted onto the surface. Because it's real geometry, the team can also report genuinely meaningful numbers directly from it: the relief range on the demo scene really is 0–91 meters, crater shapes are really measurable from the mesh, and the **vertical exaggeration** applied for visual clarity is itself computed (not an arbitrary "make it look dramatic" multiplier), so the 3-D view never visually overstates how bumpy the real terrain is.

Q4 — How does FastAPI's async handling deal with GPU-bound inference?

Plain-English answer: this question contains a hidden false premise, and the team's answer politely corrects it: their inference isn't GPU-bound at all — it's **CPU-bound** (see Q6 below for why). The real question worth answering is "how does the backend handle multiple people using the app at the same time without one person's request freezing everyone else's?" The answer: ordinary (`sync def`) endpoints automatically get run in Starlette's **threadpool** (about 40 worker threads), so one slow computation doesn't block the **event loop** that's trying to serve everyone else. `async def` (genuinely asynchronous code) is reserved for spots that are actually waiting on something external, like a file upload streaming in over the network. But the *real* trick for handling repeated load isn't concurrency mechanics at all — it's **caching**: the same match request is checked against an in-memory dictionary first, then an Upstash Redis cache, before ever falling through to actually re-running the expensive matching pipeline. So identical requests, however frequent, essentially never re-compute.

Q5 — Did you convert PyTorch models to ONNX yourselves? Any accuracy loss?

Plain-English answer: yes, done in-house via `torch.onnx.export`, and there's no meaningful accuracy loss because the conversion is a straightforward **float32-to-float32** translation of the exact same mathematical operations — no **quantization** (reducing numeric precision) and no **pruning** (removing parts of the network) happened. The team didn't just assume this was lossless — they wrote a verification step right into the training script that loads the exported ONNX file back up in **onnxruntime** and checks that its output still has the expected shape (a 128-number vector) and is still properly **L2-normalized** (length ≈ 1.0). Underneath, the actual model is a compact 4-layer **CNN** (see Q7/Part 3) trained with **triplet loss** to produce these 128-dimensional embeddings from synthetically shaded crater terrain rendered under randomized lighting.

Q6 — CPU or GPU (CUDA) for running the model?

Plain-English answer: deliberately CPU — the code explicitly sets `providers=["CPUExecutionProvider"]`. The reasoning: the actual per-request workload is tiny (embedding small 128×128 image patches), fast enough to finish in well under a millisecond even on an ordinary processor. Adding a GPU/**CUDA** dependency would mean every deployment environment needs specific NVIDIA hardware and drivers installed — a real practical burden — in exchange for a speed benefit nobody would ever notice at this scale. The team notes switching to GPU later, if a bigger/heavier model needed it, would be a one-line code change — they're not locked in, just not paying an unnecessary cost today.

Q7 — Which PyTorch version, extensions used?

Plain-English answer: PyTorch 2.11.0, CPU build, with no extra add-on libraries (no PyTorch Lightning, a popular training-loop-simplification library; no torchvision, a common computer-vision-specific add-on). The training loop (the code that actually feeds data through the model and updates its weights) was hand-written rather than using a pre-built framework, using the **Adam optimizer** at a learning rate of 1e-3 (0.001) on batches of 8 training triplets. Everything around loading/preparing the training data itself uses just NumPy and PIL — no specialized data-pipeline library. The team also notes the exact same script runs fine on a laptop CPU (for the demo-scale model) and is ready to run unmodified on a free Kaggle GPU (a "T4") if training a larger, full-scale version of the model later.

Q8 — Why Supabase instead of running raw PostgreSQL yourself?

Plain-English answer: Supabase gives them the full power of real SQL (including flexible `jsonb` columns for storing things like geographic footprint shapes and 3×3 homography matrices) plus a ready-made REST API, without the team having to install, configure, secure, and maintain a database server themselves. On the security side, they're explicit that **Row-Level Security (RLS)** is genuinely turned on for all four of their tables (verifiable directly in `supabase/schema.sql`, not just claimed) — the `scenes` and `metrics` tables allow public *reading*, but the `jobs` and `matches` tables have **no** public access policy at all; those can only be written to using the privileged **service-role key**, which lives only in a backend `.env` file that's explicitly excluded from version control (`.gitignore`) and never reaches the browser.

Q9 — Does Supabase Storage handle large satellite images (GeoTIFFs) well?

Plain-English answer: an unusually candid "no, and that's fine, because we don't ask it to." Multi-megabyte satellite images never go anywhere near the database layer at all — the actual processed image files, elevation grids, and crater data live as ordinary files on the server's own disk (organized per-scene, and excluded from version control since they can always be regenerated from source data). Only small, genuinely database-appropriate data — JSON rows, with keypoint samples explicitly capped at 500 entries — actually goes into Supabase. The team is upfront that in a real production deployment, this would instead be handled by NASA/ISRO's own archival systems (**PDS**) plus dedicated large-file object storage, not a general-purpose database at all. The same file-size discipline is applied to the Redis cache too — Upstash caps individual values around 1 MB, so the code deliberately trims large payloads before caching them.

Q10 — Is Upstash's HTTP-based Redis fast enough for real-time job queuing?

Plain-English answer: the team sidesteps the premise again — they never use Redis as a job queue in the first place. They use it for exactly the three things HTTP-based Redis is genuinely well-suited for: caching computed results, tracking job status, and enforcing rate limits — all of which comfortably tolerate the roughly 5–20 millisecond latency of an HTTP round-trip, since they sit in front of a pipeline that already takes multiple seconds to compute anyway (so a few extra milliseconds of cache-lookup overhead is irrelevant). If they *did* need true real-time job queuing, they note they'd reach for a traditional (non-HTTP) Redis connection instead — and because all their Redis interactions are already funneled through four small helper functions in one file (`integrations.py`), making that swap later would be a small, contained change.

Q11 — Which Redis data structures are used for job management?

Plain-English answer: just two, deliberately kept minimal. **Strings** (via plain `SET`/`GET` commands, with a **TTL** expiration) handle the match-result cache and simple job-status flags. A **sorted set** implements the narration rate limiter — using the sequence `ZADD` (record this request's timestamp) → `ZREMRANGEBYSCORE` (drop any recorded timestamps older than the current 1-minute window) → `ZCARD` (count how many timestamps remain) → `PEXPIRE` (make sure the whole tracking key eventually cleans itself up). This specific pattern implements a true **sliding window** rate limit (always "the last 60 seconds from right now"), which is more robust against burst-abuse than a simpler fixed-window counter. The team explicitly didn't reach for Redis's other features — lists, streams, pub-sub — because nothing in the app actually needs them; using only the data structures a problem genuinely calls for is presented as a deliberate design discipline, not an oversight.

Q12 — Is Sentry used on the frontend, backend, or both?

Plain-English answer: both, tracked as two separate Sentry projects. The browser gets Sentry's JavaScript SDK, configured with a **DSN** kept right in `frontend/config.js` — safe to have visible in browser code, since a DSN only allows *sending* error reports in, not reading anything out. The Python backend uses Sentry's Python SDK, with **breadcrumbs** recorded after each stage of the processing pipeline, so if something does eventually fail, the team can see the exact sequence of steps that led there. There's even a dedicated `/debug/sentry-test` endpoint whose only job is to deliberately trigger a test error, letting the team confirm error reporting is actually working end-to-end (rather than just trusting it silently).

Q13 — Why Gemini instead of GPT or Claude?

Plain-English answer: three concrete reasons, in order. **Cost** — Gemini's free tier requires no credit card at all, which matters because the team's stated goal is a \$0 total running cost for the whole project. **Latency** — the actual task (turning a handful of already-computed numbers into a summary of at most 120 words, in a single request with no back-and-forth) is small enough that a lighter, faster "flash-lite" tier model is more than sufficient; a bigger model's larger **context window** buys nothing here since the input is tiny regardless. **Framing/safety** — the model is deliberately **prompt-locked** to only narrate metrics the pipeline already computed, never to invent its own numbers, which matters more than which specific AI brand is doing the narrating.

Q14 — What's the fallback if Gemini goes down or gets rate-limited?

Plain-English answer: three independent layers of protection, all actually tested working, not just theoretical. **(1)** If Gemini can't be reached, a **local-template** narration kicks in instead, built from the exact same computed metrics — and the interface honestly labels which source (`gemini` vs `local-template`) actually produced the text shown, rather than pretending it's always the AI. **(2)** If Gemini responds with a **429** (rate-limited) error, the code enters a 5-minute **cooldown** and remembers that state, so a temporarily-exhausted quota produces one single warning notice, not a fresh error every time someone clicks the narrate button. **(3)** On top of all that, a Redis **sliding-window** rate limiter (20 requests/minute) sits upstream of the whole narration feature, throttling usage proactively before it could ever hit Gemini's own limits in the first place.

Q15 — What would block a fully offline / on-premise ISRO deployment?

Plain-English answer: nothing structural — every third-party integration is built as a **no-op** if its access key/configuration is simply absent, and each one has a documented swap path to a fully self-hosted equivalent: Supabase → plain PostgreSQL (their schema file is already ordinary SQL, no Supabase-specific magic), Upstash → a locally-run Redis server (their four helper functions would just point at a local connection instead of an HTTP endpoint), Sentry → a self-hosted open-source alternative (GlitchTip) or simply turned off, Gemini → the built-in local-template narration, or a locally-run open-source language model instead. Crucially, the actual mission data the pipeline depends on — imagery products in their native archival formats, satellite reference imagery — is already stored and processed entirely on local disk; nothing about the *core* pipeline requires reaching out to the internet at all.

Q16 — Walk through the live deployment

Plain-English answer, piece by piece:

Frontend on Vercel: since the whole UI really is just one static HTML file with no build step, the thing Vercel actually serves is literally the exact source file — `vercel.json` just tells Vercel which folder that file lives in. Vercel's value-add here is purely its global **CDN**, making that one file load fast worldwide.

Backend on Render: the backend can't be a simple static file or a typical serverless function, because it needs Python, OpenCV, and access to real scene data files on disk — so it runs as a genuine, always-running **web service**, with its build/start commands and health check fully described in one `render.yaml` file (a "Blueprint"), so a brand-new deployment is entirely automatic and reproducible. Because the demo scenes and the trained ONNX model file are committed directly into the repository, a completely fresh deployment boots up already fully functional — nothing extra needs to be downloaded or trained first.

The connection between them: a single configuration value (`window.API_BASE`) in the frontend tells it where the backend lives; the backend allows requests from any origin (**CORS** `allow_origins=["*"]`) since there's no private user data to protect. There are no login **cookies/sessions** — every request is **stateless**, standing entirely on its own.

Cost: genuinely \$0, using each service's free tier. The trade-off of the free tier is that the backend goes to sleep after roughly 15 minutes of inactivity (a **cold start**), taking about 50 seconds to wake back up on the

very next request — a one-time inconvenience, not an ongoing performance problem, since the actual pipeline itself runs in seconds once warm, and repeated identical requests hit the cache anyway.

Why this setup proves the system is genuinely portable, not just “happens to work on our specific cloud accounts”: the exact same backend code boots up unmodified whether it’s run via `uvicorn` on someone’s personal laptop, on Render, or (per Q15) on real ISRO infrastructure — nothing about the core logic assumes any particular hosting environment.

Demo-day practical note: for the actual live demo, the team runs the frontend from Vercel as usual, but points it at the backend running on their *own* laptop (made publicly reachable via a temporary **HTTPS tunnel**), specifically so there’s no cold-start wait and their full local reference-imagery library (3.8 GB) is instantly available for live NASA auto-selection on any freshly uploaded image — with the always-on Render deployment kept as a reliable fallback URL in case anything goes wrong with the laptop/tunnel setup.

4. Part 2 elaborated — the exhaustive system walkthrough

This section is the “what actually happens, in order” story — from the moment a judge opens the page to every button they might click.

① Site opens

The Python backend (running via `uvicorn`) itself serves up the single frontend HTML file when a browser requests the homepage. That file loads `config.js` (the small file holding the backend’s address, the Sentry DSN, and a shared helper function called `fetchT` that wraps every network request with a hard timeout — meaning no request can silently hang forever with no feedback to the user). The **Sentry** browser error-tracker initializes, the **import map** loads the Three.js 3-D library from its CDN, and the page immediately pings `GET /health` — turning a status badge green and confirming the backend reports it has both the classic SIFT matcher and the custom learned ONNX descriptor ready to go. It then fetches `GET /craters`, which populates the scene-selection dropdown menu (each option’s label/details come straight from that scene’s own `metadata.json` file — its product ID, sun angle at capture time, geographic footprint corners, and elevation range). Finally, the page automatically runs a match on whichever scene is selected by default, so a judge sees real results immediately without needing to click anything first. Every single one of these steps has its own timeout, so if any step is slow or fails, the user sees a visible error/retry state rather than a page that just looks frozen.

② MATCH — the core matching pipeline

Hitting `GET /match/{scene_id}` triggers the **cache-aside** pattern described earlier: check the in-memory dictionary first, then check Upstash Redis (`GET match:{id}`), and only if both come back empty does the system do the real work. That real work is: load the source image, the auto-selected reference image, and the elevation grid from disk → apply **CLAHE** contrast enhancement → run **SIFT** to get up to 8,000 keypoints (filtered by the **Low ratio test** at a 0.8 threshold) → separately run the **learned ONNX branch** (embed the top 1,500 keypoints from each image into 128-dimensional vectors, then keep pairs whose **cosine similarity** exceeds 0.5 *and* which are each other’s **mutual nearest neighbor**) → **union** both sets of candidate matches

together (so the final match list benefits from both SIFT's pixel-level geometric precision *and* the learned model's resistance to lighting differences) → run **RANSAC** with a **DLT** homography fit (3-pixel error threshold, 99% confidence, and an iteration count mathematically derived from the data itself, not hard-coded) → compute the final **RMSE** → run the **Fourier-Mellin** transform as an independent diagnostic cross-check → save everything (both to the Redis cache and as rows in Supabase). The frontend then draws the actual keypoint and inlier/outlier overlays directly onto the two images, fills in a table of detected craters, and updates a caption showing *which* reference image was automatically chosen for this match.

③ Reference auto-selection — no manual step, anywhere

For the pre-baked demo scene, the preprocessing script already searched all 8 available NASA reference image strips, looking for wherever the Chandrayaan-2 crop best lines up (scored by post-alignment **NCC**, normalized cross-correlation) — and it settled on one specific strip (`M1249388815LC`) as the best match. For any brand-new upload during a live demo, the exact same search process runs live, in real time, via `ingest.auto_select_reference()`. If, for some uploaded image, genuinely no real NASA reference strip overlaps that location at all, the system doesn't just fail — it automatically manufactures a **simulated** second "pass" of the same image (applying a slightly different brightness curve, a small rotation, a small scale change, and some added radiance gradient/noise), specifically so that *every* uploaded scene, no matter what, ends up with something matchable to demonstrate the pipeline on. Whichever path was actually taken gets honestly recorded in that scene's metadata file, and the frontend caption reads directly from that record — so a judge can always tell whether they're looking at a real independent satellite match or a simulated stand-in.

④ UPLOAD — a plain photograph

A judge can upload any ordinary image file. The backend endpoint accepts the file plus sun azimuth/elevation angles (which can be automatically inferred from mission metadata files, if provided, rather than requiring manual entry). The image is decoded, resized to a standard 1024×1024 working resolution, run through the **shape-from-shading** math (Part 4) to produce a terrain payload, and a quick client-side **hillshade** preview (a fast, simplified lighting simulation) is drawn immediately in the browser for instant visual feedback. From there, the judge can either send the result straight to the 3-D terrain viewer, or go further and have the system treat it as a brand-new full "scene" — running crater detection, auto-selecting a reference image, and performing full registration — after which it appears as a selectable option back in the main MATCH tab, just like the pre-baked demo scenes.

⑤ UPLOAD — proper satellite data formats

Beyond plain photos, the system also accepts real mission data file formats directly: an `.xml` + `.img` file pair, or a `.zip` archive containing them. The backend's file-reading code understands both ISRO's PDS4 format and NASA's older PDS3 format (including correctly skipping past PDS3's specific 5,064-byte binary header before reading the actual 16-bit integer image data), memory-maps the (potentially huge) file efficiently rather than loading it all into RAM at once, automatically finds and crops out the most information-rich 1024×1024 region of the image, and then runs through that exact same shape-from-shading/crater-detection/auto-reference pipeline as any other upload.

⑥ TERRAIN — building the 3-D view

Hitting `GET /terrain/{id}` loads the raw elevation grid from disk, applies the **truncated Fourier low-pass** smoothing (Part 4) to get a clean 192×192 working grid, runs the **marching squares** algorithm at 8 different elevation levels to generate contour line data, and packages everything as JSON for the frontend. The frontend's `buildTerrain()` function then turns that data into the actual visible scene: the 72,962-triangle mesh, the wireframe overlay, the 3-D contour polylines placed at their correct heights, a computed vertical exaggeration factor, and interactive **OrbitControls** so a judge can freely rotate/zoom the model themselves.

⑦ NARRATE — the AI-generated summary

Hitting `GET /narrate/{id}` first checks the Redis sliding-window limiter (20 requests/minute), then compiles a summary made *only* from values the pipeline already computed (never anything invented on the spot), sends that to Gemini (specifically the `gemini-flash-lite-latest` model) with instructions to produce a natural-language summary of at most 120 words, suitable for reading aloud to judges. If Gemini is unavailable or rate-limited, the local-template fallback (Q14) takes over instead, and a 429 response triggers the cooldown logic.

⑧ Error handling, end to end

Whenever something does go wrong, **Sentry** captures it — as two entirely separate tracked projects (one for frontend JavaScript errors, one for backend Python errors), each enriched with the **breadcrumb** trail of pipeline stages leading up to the failure. For deeper debugging beyond just the error message, every intermediate artifact from every processing stage is also saved directly to disk (per-scene folders, plus a master `registry.json` index file), so the team can inspect exactly what the pipeline actually produced at each step, not just the final result.

5. Part 3 elaborated — training the ONNX descriptor, step by step

Why this exists at all: classic feature matchers like SIFT are excellent at finding distinctive points in an image, but their numeric "fingerprint" for each point can be thrown off by big lighting changes — exactly what happens when comparing two satellite photos of the same crater taken at different times with the sun in a different part of the sky. The team's response was to train a small custom neural network whose entire *job* is to produce a fingerprint for a small image patch that stays consistent *even when the lighting changes*, as a second, complementary matching signal alongside SIFT.

Step 1 — Generating synthetic training terrain (no manual labeling needed). Because there's no large public dataset of "here are two lunar photos of the exact same crater under different lighting, hand-labeled as matching," the team sidesteps the need for real labeled data entirely by procedurally generating their own crater-like terrain from a formula: a parabolic bowl shape (the crater's depression) plus a raised ring around its edge (the rim) plus, optionally, a small central peak (common in larger real craters) plus multi-scale random roughness (**fractional Brownian motion**, layering 5 different scales of randomness together for a natural,

non-repetitive texture) — with a separately-generated random reflectivity ("**albedo**") pattern layered on top, so brightness variation isn't only coming from shape.

Step 2 — Rendering shaded image pairs at randomized sun angles (the actual research problem, baked directly into the training data). For each generated terrain, the team renders it as a realistic-looking shaded image using the same **Lambertian reflectance** physics described in Part 4, but does this *twice* per terrain, using two genuinely different, randomized sun positions each time — plus once more for an entirely *different* terrain shape, to serve as a clearly-wrong example. This produces exactly the three-way structure a **triplet loss** needs: an "anchor" image, a "positive" image (same real terrain, different lighting — the hard case the network needs to learn to still recognize as the same place), and a "negative" image (genuinely different terrain, used to teach the network what "different" should look like). Because the *only* thing consistently different between the anchor and the positive is the sun angle, the network is structurally forced to learn to identify actual terrain shape and ignore illumination — precisely the failure mode plain SIFT struggles with.

Step 3 — The network architecture. A compact 4-layer **CNN** ("PatchEncoder"): a first convolution layer expanding from 1 grayscale channel to 16 learned feature channels, a ReLU activation and a max-pooling downsampling step, then further convolution layers deepening to 32, then 64, then 128 channels, an adaptive average-pooling step to collapse everything down to a fixed size regardless of input size, and a final linear layer producing a 128-number output vector, which is then **L2-normalized** (rescaled to have a length of exactly 1). The result is a compact, comparable "fingerprint" for any 128×128 image patch.

Step 4 — The loss function (what the network is actually optimized to minimize).
$$L = \text{mean}(\text{clamp}(\|a-p\|_2 - \|a-n\|_2 + m, 0))$$
 — reading this piece by piece: $\|a-p\|_2$ is the (Euclidean/L2) distance between the anchor's embedding and the positive's embedding; $\|a-n\|_2$ is the distance between the anchor's embedding and the negative's embedding; the network is penalized whenever the *positive* distance isn't smaller than the *negative* distance by at least the safety **margin** m (set to 0.2); $\text{clamp}(\dots, 0)$ means any already-satisfied case (where the network is doing fine) contributes zero penalty rather than a negative "reward," so training focuses entirely on cases still being gotten wrong; and $\text{mean}(\dots)$ averages this penalty across every triplet in a training batch. In plain terms: "*make same-terrain-different-lighting pairs meaningfully closer together than different-terrain pairs, with a real safety margin, not just barely.*" Training itself uses the **Adam** optimizer at a 0.001 learning rate, in batches of 8 triplets at a time.

Step 5 — Validating before trusting the model. Before exporting anything, the training script evaluates the trained network on 100 brand-new random triplets it never saw during training, and reports a **ranking accuracy** — literally, what fraction of the time the positive really did end up closer to the anchor than the negative did. The team's stated targets are "better than random guessing" (above 0.5) at minimum, ideally well above 0.9 — and critically, this number is genuinely printed by the script and checked, not just assumed to be fine.

Step 6 — Exporting to ONNX. Using `torch.onnx.export` with **opset version 17** and a **dynamic batch axis** (so the exported model can process anywhere from 1 to hundreds of patches per call without needing separate exports for each batch size) — and, as emphasized repeatedly, with no quantization or pruning, meaning it's a precision-preserving, op-for-op translation of the exact same trained model.

Step 8 — How the model is actually used at inference time. For each of the two images being compared, the pipeline takes their top 1,500 SIFT keypoints, crops out a small 128×128 pixel patch around each one (padding at image edges as needed), runs all those patches through the trained model in efficient batches, computes the **cosine similarity** between every possible pair of embeddings from image A versus image B, keeps pairs above a 0.5 similarity threshold that are also each other's **mutual nearest neighbor**, and finally unions those matches together with the separate SIFT-based candidate matches before they all go into RANSAC together. And critically for demo reliability: if this whole learned-model step were ever to fail for any reason, the code is written to gracefully degrade — falling back once to plain SIFT-only matching with a logged warning, rather than crashing the request. The live demo literally cannot return a server error just because of a model-inference hiccup.

6. Part 4 elaborated — every formula, decoded

Each subsection below follows the same structure: **what it's computing**, **what each symbol means**, and **why this particular equation is the correct/sensible one to use**.

6.1 Shape-from-shading (turning one 2-D photo into a 3-D height map)

The physics equation:


$$I(x,y) = A \cdot (-p \cdot s_x - q \cdot s_y + s_z)$$

What it's saying: $I(x,y)$ is the brightness recorded at pixel position (x,y) in the photograph. A is the surface's overall reflectivity (**albedo**). (s_x, s_y, s_z) is the direction the sunlight is coming from, expressed as a 3-D unit vector (a direction with length exactly 1) in an (east, south, up) coordinate frame — this is computed from the sun's compass azimuth and elevation angle at the time the photo was taken. (p, q) is the local terrain **slope** at that pixel — specifically, how steeply the ground tilts in the x-direction and in the y-direction, respectively (mathematically, the **partial derivatives** $\partial h / \partial x$ and $\partial h / \partial y$ of the unknown height function h). In plain words: *"how bright a point looks depends on how directly its local slope faces the sun."* This is the standard **Lambertian reflectance** model — the same basic physical assumption behind why a photo of a ball looks shaded the way it does.

The core problem — and the honest fix. A single 2-D photo only gives you *one* brightness number per pixel, but the true slope has *two* unknown components (p and q) — so this equation alone is mathematically underdetermined; you can't solve for a unique slope from brightness alone using just this one equation. Specifically, a single image can only ever pin down the component of slope that runs *parallel to the sun's horizontal direction* — slope running perpendicular to the sun's direction leaves brightness completely unchanged, so there's genuinely no information about it in the photo. Faced with that ambiguity, the team makes a deliberate, principled choice: solve for the **minimum-norm solution** — among all the (infinitely many) slope fields that would produce the observed brightness equally well, pick the smallest, most conservative one, rather than guessing arbitrarily large slopes in the direction the data can't actually constrain.

The practical computation:



$$t = -\Delta I / (a \cdot \|s_h\|^2), \quad (p, q) = -(s_x, s_y) \cdot t$$

Here $\boxed{a}$ is a robust estimate of the "brightness-if-facing-straight-at-camera" baseline (computed as the median brightness divided by $\boxed{s_z}$), $\boxed{\Delta I}$ is how much each pixel's brightness differs from that baseline, and $\boxed{s_h}$ is just the horizontal (east/south) part of the sun direction vector. This directly produces the slope estimate $\boxed{t}$ along the sun's horizontal direction, which then gets split back out into the p and q slope components. Two safety measures are applied here too: extreme, physically implausible slope values are clipped to a maximum of 35°, and pixels sitting in deep shadow (the darkest 1% of the image) are deliberately zeroed out, since a shadow, by definition, carries no usable brightness-vs-slope information at all — trying to extract a slope estimate from pure shadow would just be extracting noise.

Turning slopes into an actual height map (the Poisson solve):



$$\nabla^2 h = \partial p / \partial x + \partial q / \partial y \Rightarrow \hat{h}(k) = F[\text{div}](k) / (-k^2)$$

You now have a slope (steepness) value at every pixel, but what you actually want is *height*, which means you need to effectively "add up" (integrate) all those slopes across the whole image consistently. This is a classic mathematical problem called a **Poisson equation**, and it's solved here using the **FFT**: converting the slope data into the *frequency domain* turns this "adding up slopes consistently across a whole 2-D grid" problem into simple, direct division for every frequency component ($\boxed{k^2}$ represents each frequency's "wavenumber," essentially how fine/coarse that particular repeating pattern is) — a standard, very fast numerical technique for solving exactly this class of problem. Two small technical safeguards: the very lowest frequency component (the "**DC**" term, essentially "what's the overall average height") is mathematically undefined in this formula (it would require dividing by zero) and is manually guarded against, and a **Hanning window** smooths the image's edges beforehand, to prevent the FFT from misinterpreting the sharp cutoff at the image boundary as a real repeating pattern.

The honesty step — calibrating to a plausible absolute scale. Because a single image, as established, can never pin down absolute height (only relative shape, entangled with unknown albedo), the reconstructed

relief is deliberately rescaled so its overall **RMS** (root-mean-square, i.e., "typical") slope steepness matches a stated, physically reasonable assumption of 8° — and this scaling factor is explicitly written back out into the scene's metadata file, reported as what it is (a calibration assumption), never silently presented as if it were an independently-measured fact. The guide's own phrase for this — "honesty is part of the math" — is really about the team choosing to be transparent about a real, unavoidable physical limitation rather than hiding it.

6.2 Crater detection and depth estimation

Circular crater shapes are located using the **Hough circle transform** (searching for radii between 5 and 90 pixels) on a **CLAHE**-enhanced version of the elevation grid (boosting local contrast makes faint crater rims easier to detect). Once a crater's shadow is identified, its depth is estimated with a simple, physically-grounded approximation: $\text{depth} \approx L \cdot \tan(\theta)$, where L is the measured length of the shadow cast by the crater's rim and θ is the sun's elevation angle at capture time — basic trigonometry (a shadow's length relative to an object's height depends directly on how high the sun is in the sky), assuming a simplified flat crater floor. Each detected crater's pixel position is then converted into real selenographic (lunar) latitude/longitude coordinates using **bilinear interpolation** against the mission's own geometry reference data.

6.3 Candidate keypoint matching (combining two independent signals)

SIFT contributes its own candidate matches, filtered by the **Lowe ratio test** ($d_m < 0.8 \cdot d_n$) — the best match's distance/dissimilarity score must beat the second-best candidate's score by a comfortable margin, or it's discarded as too ambiguous). Separately, the learned branch computes a full similarity matrix $S = E_1 E_2^T$ (every embedding from image 1 compared against every embedding from image 2, all at once, via matrix multiplication — this is exactly **cosine similarity**, since the embeddings are already L2-normalized), keeping pairs above a 0.5 similarity threshold that are also **mutual nearest neighbors**. These two independently-produced candidate lists are then simply **combined (unioned)** — the guide's framing is that SIFT contributes fine geometric/pixel-level precision, while the learned branch contributes resilience to lighting differences, so the union is genuinely stronger than either alone.

6.4 RANSAC with a mathematically-derived iteration budget

A **homography** (a 3×3 matrix describing how one flat image's viewpoint maps onto another's) is fit inside **RANSAC**, using the **DLT** algorithm on the minimal sample size of 4 matched points, with a 3-pixel error tolerance and a 99% confidence target.

The key formula (the "Problem 3" the guide references):

$$s \geq \log(1 - p) / \log(1 - w^s), \quad s = 4$$

This computes exactly how many random-sample trial rounds ($\lfloor k \rfloor$) RANSAC needs to run to reach its target confidence $\lfloor p \rfloor$ (0.99, i.e., 99% sure it found the *actual* correct transformation, not one that got lucky fitting mostly-wrong data). $\lfloor w \rfloor$ is the *current estimated inlier fraction* — roughly, "what fraction of all candidate matches are actually correct" — and $\lfloor s \rfloor$ is the minimal sample size (4, for a homography). The intuition: if a

large fraction of your candidate matches are already good (w close to 1), you need relatively few random trials to have good odds of accidentally drawing 4 good ones together; if most candidates are noisy/wrong (w small), you need many more attempts before you get lucky. What matters most here is that **this number is computed live from the data's own measured quality, every single time**, rather than being a fixed guess baked into the code — a genuinely more principled (and more defensible, to a judge asking "why that number?") approach than hard-coding, say, "always run 1,000 iterations."

Measuring the final fit's quality:



$$\text{RMSE} = \sqrt{\text{mean } \|H \cdot x_i - x_i'\|^2}$$

Take every point RANSAC decided was a genuine **inlier**, apply the found homography H to its position in image A, and measure how far off that lands from its actual matched position in image B — square each of these gaps, average them, and take the square root (this is the **RMSE**, in pixels). A smaller RMSE means the found geometric transformation more precisely explains the actual matched points.

Why RMSE eventually stops improving — and why that's not a bug. The guide invokes the **Cramér-Rao bound**, a foundational statistics result stating that $\text{Var} \geq \sigma^2 / \sum (dI/dx)^2$ — in plain terms, there is a hard mathematical floor on how precisely *any* possible method could ever pin down a position, given how noisy the image data is and how sharply brightness actually changes across it. Once RMSE plateaus, it's not that the matching algorithm has a flaw to be fixed — it's that the *photograph itself*, at its actual noise level and actual sharpness, simply doesn't contain any more extractable positional information to squeeze out. This is a genuinely strong, physically-grounded answer to have ready if a judge pushes on "why isn't your RMSE lower?"

6.5 Fourier-Mellin registration (an independent cross-check)

This is a classic technique (the **Reddy-Chatterji** method) for estimating rotation, scale, and translation between two images purely from their frequency content, used here purely as a secondary diagnostic — not the primary matching path (SIFT + RANSAC remains the actual precise registration method).

How it works: take the log-magnitude of each image's Fourier transform ($A = \log|\text{FFT}(a)|$), re-map that from ordinary x/y coordinates into **log-polar coordinates** (360 angle bins $\times$ 256 log-scaled radius bins). This specific re-mapping is the clever trick: a *rotation* of the original image becomes a simple *sideways shift* along the angle axis in this new representation, and a *scale change* becomes a simple shift along the log-radius axis — turning two originally-tangled unknowns (rotation and scale) into two independent, simple shift-detection problems.



$$(\Delta z, \Delta \ln \rho) = \text{argmax}_{\mathcal{F}^{-1}} (P_A \cdot P_B^* / |P_A \cdot P_B^*|)$$

This is **phase correlation** — a standard, robust way to detect exactly how much one signal is shifted relative to another using frequency-domain phase information (rather than comparing raw pixel brightness directly,

which is much more sensitive to noise and illumination differences). The two shift amounts found this way directly give the rotation angle θ and scale factor s . Once the images are computationally de-rotated and de-scaled to match, a *second* phase correlation pass then finds the remaining plain translational offset between them.

6.6 Mutual information (a reserved, not-yet-primary tool)


$$I(A;B) = \sum p(a,b) \cdot \log[p(a,b) / (p(a)p(b))]$$

This measures how statistically related two images' brightness values are — $p(a,b)$ is the *joint* probability of seeing brightness value a in image A at the same location where image B shows brightness value b ; $p(a)$ and $p(b)$ are each image's *individual* brightness probabilities on their own. If knowing image A's value tells you nothing extra about image B's value at that spot (they're statistically independent), this whole sum works out to exactly zero; the more the two are related — in *any* kind of way, not just simple proportionality — the higher this value climbs. It's guaranteed to never go negative, a fact that follows mathematically from **Jensen's inequality** (a general property of "concave," frown-shaped functions like the logarithm). The guide notes this tool is implemented and ready, reserved specifically for comparing genuinely different sensor types (an "IIRS stretch goal" — IIRS presumably referring to a different Chandrayaan-2 instrument with fundamentally different imaging physics than OHRC), where a simpler correlation measure wouldn't be reliable, since the relationship between two different sensors' brightness values isn't necessarily a simple straight line.

6.7 Truncated Fourier surface (smoothing the terrain mesh)


$$z(x,y) = \sum c_{mn} \cdot \exp(i2\pi(mx/Lx + ny/Ly))$$

This is the standard mathematical fact that *any* 2-D surface can be exactly represented as a sum of many repeating wave patterns at different frequencies, each with its own coefficient c_{mn} (computed via a 2-D **FFT**). The practical trick: keeping *only* the coefficients for low m and n (the coarse, large-scale, slowly-varying patterns) and simply discarding the rest is an exact, mathematically clean **low-pass filter** — the big-picture terrain shape survives essentially untouched, while fine pixel-level noise (which lives in the high-frequency terms) is removed. Handily, the guide notes that a specific repeating stripe pattern sometimes present in real satellite scan data (a **scan-line artifact**, caused by the physical way the camera captures the image line-by-line) shows up concentrated in one very specific (m, n) frequency bin — so it can be surgically zeroed out from that one spot, with no need for any separate, more complex artifact-removal step. This resulting coefficient grid isn't just used for visualization — it directly becomes the actual 3-D mesh's input data.

6.8 Marching squares (drawing contour lines precisely)

For a contour line at some target elevation value z_k , whenever that value falls between two neighboring grid

points' values (say z_1 and z_2) along one edge of a grid cell, the exact crossing point is found by simple linear interpolation:


$$t = (z_k - z_1) / (z_2 - z_1), \quad P = P_1 + t \cdot (P_2 - P_1)$$

t tells you *how far along* that edge (as a fraction from 0 to 1) the contour actually crosses, and P is the resulting precise sub-pixel position. Doing this consistently for every grid cell, and connecting the resulting crossing points according to each cell's specific pattern of "which corners are above/below the target level" (the same standard approach used by the well-known `skimage.find_contours` library function), produces smooth, sub-pixel-accurate contour lines rather than blocky, grid-aligned steps.

6.9 Reference auto-selection statistics

Finding the best-matching NASA reference strip starts with a fast, coarse search: **NCC** (normalized cross-correlation, specifically OpenCV's `TM_CCOEFF_NORMED` scoring method) run on a shrunk-down (8× smaller) preview of each candidate reference strip, which is cheap enough to check against all 8 available strips quickly. Whichever strip scores best there then goes through a full, precise registration pass (a coarse translation alignment, followed by full SIFT + RANSAC homography refinement, as described above) — and the strip with the best *final*, post-alignment NCC score wins (for the baked demo scene, that turned out to be strip `M1249388815LC`). Separately, as an honest "how well do these two completely independent satellites actually agree" sanity check, the team computes both **Pearson correlation** and **gradient-magnitude correlation** between the NASA reference image and the independently-derived shape-from-shading terrain from the Chandrayaan-2 image — and they're explicit that these agreement numbers are recorded and reported *honestly*, including when they come out fairly low, since two different missions imaging the same spot under very different sun geometries genuinely *are* expected to disagree somewhat — that expected disagreement, and how to still extract reliable matches despite it, is the core research problem this whole project is built around.

7. Quick-reference numbers

Number	What it is
36,864	Vertices in the 3-D terrain mesh
72,962	Triangles in the 3-D terrain mesh
191×191	Mesh subdivision segments (<code>PlaneGeometry</code>)
192×192	Elevation (DEM) grid resolution used to build the mesh
0–91 m	Measured relief range on the demo (OHRC) scene

Number	What it is
8	Number of marching-squares contour levels drawn
~40	FastAPI/Starlette threadpool worker count
8,000	Max SIFT keypoints extracted per image
0.8	Lowe ratio test threshold
1,500	Top keypoints per image sent into the learned ONNX branch
128 × 128	Pixel size of each patch fed to the learned model
128-D	Output embedding size of the learned model
0.5	Cosine-similarity threshold for a learned-branch candidate match
3 px	RANSAC reprojection error threshold
0.99	RANSAC target confidence (p)
4	Minimal sample size for a homography fit (s)
8	Number of LRO NAC reference strips searched for auto-selection
M1249388815LC	The NASA strip auto-selected as the demo scene's best reference
500	Max keypoint samples stored per match in Supabase
~1 MB	Upstash Redis per-value size cap
20/min	Narration rate limit (sliding window)
5 min	Gemini 429 cooldown duration
≤120 words	Max length of the AI-generated narration
17	ONNX opset version used for export
2.11.0	PyTorch version (CPU build)
0.2	Triplet loss margin
1e-3	Adam optimizer learning rate
8	Training batch size (triplets)

Number	What it is
100	Held-out triplets used for validation accuracy
0.5 / 0.9	Minimum-acceptable / target ranking accuracy
5–20 ms	Typical Upstash HTTP-Redis round-trip latency
~15 min	Render free-tier idle time before sleep
~50 s	Render free-tier cold-start wake time
3.8 GB	Local NASA reference-imagery library used at demo time
5,064 bytes	PDS3 attached-header size skipped when parsing NASA <code>.img</code> files
\$0	Total infrastructure cost of the entire deployed stack

8. Closing note

Every explanation above was written to trace back to something explicitly stated in the original `PRESENTATION_GUIDE.md` — no new claims about the system were invented here, only unpacked. If a judge asks *why* a specific number or formula was chosen, the original guide's own answer is almost always "because it's derived from the data itself, not hard-coded" (the RANSAC iteration budget, the vertical exaggeration, the relief calibration, the sun-angle auto-fit) — that recurring theme of "*the math is honest about its own assumptions and limits*" is arguably the single strongest thread running through the entire project, and it's worth having ready as an answer in its own right if a judge ever asks "what's the one thing you're proudest of technically?"